

Everything in this program should look familiar, because we've used it all in the previous two chapters. The main difference here is that we can print the formatted response string ❶ instead of writing it to a file, because the output is not particularly long.

The output is a dictionary of information about the article with the ID 31353677:

```
{
  "by": "sohkamyung",
❶ "descendants": 302,
  "id": 31353677,
❷ "kids": [
    31354987,
    31354235,
    --snip--
  ],
  "score": 785,
  "time": 1652361401,
❸ "title": "Astronomers reveal first image of the black hole
    at the heart of our galaxy",
  "type": "story",
❹ "url": "https://public.nrao.edu/news/.../"
}
```

The dictionary contains a number of keys we can work with. The key "descendants" tells us the number of comments the article has received ❶. The key "kids" provides the IDs of all comments made directly in response to this submission ❷. Each of these comments might have comments of their own as well, so the number of descendants a submission has is usually greater than its number of kids. We can see the title of the article being discussed ❸ and a URL for the article being discussed as well ❹.

The following URL returns a simple list of all the IDs of the current top articles on Hacker News:

<https://hacker-news.firebaseio.com/v0/topstories.json>

We can use this call to find out which articles are on the home page right now, and then generate a series of API calls similar to the one we just examined. With this approach, we can print a summary of all the articles on the front page of Hacker News at the moment:

```
hn
_submissions.py from operator import itemgetter

import requests

# Make an API call and check the response.
❶ url = "https://hacker-news.firebaseio.com/v0/topstories.json"
r = requests.get(url)
print(f"Status code: {r.status_code}")

# Process information about each submission.
❷ submission_ids = r.json()
```